PRÁCTICA N.º 9 — HOJA DE EVIDENCIAS

Despliegue en Kubernetes con Helm y GitOps

COM-600 · Microservicios · Gestión 2/2026 · Tema 7 — Kubernetes, Estrategias de Despliegue y GitOps

Nombre y apellidos	David Franz Soliz Ortega
Código universitario (CU)	35-5261
Carrera y paralelo	Ing. de sistemas
Fecha de entrega	21/10/2026
Sistema operativo y versiones de Docker, kubectl, k3d y Helm	Windows 10 64bits, Docker version 29.7.2, build a7dcaa6
Repositorio (URL)	https://github.com/David-Franz-Soliz-Ortega/MicroServiciosDFSO/tree/27a450b9090811bd961335d0fbebe857ab15ef50/practicas/practica%209

Cómo se llena: Este documento se completa mientras se ejecuta la Parte 1 del enunciado. Cada captura y cada pregunta lleva el mismo número aquí y allá.

Las capturas deben ser de su propia máquina: debe verse el nombre de usuario o del equipo en la terminal. Recorte solo lo necesario, pero que el comando y su salida se lean con claridad.

Al terminar, exporte este archivo a PDF y súbalo a la plataforma junto con el enlace del repositorio.

Control de avance

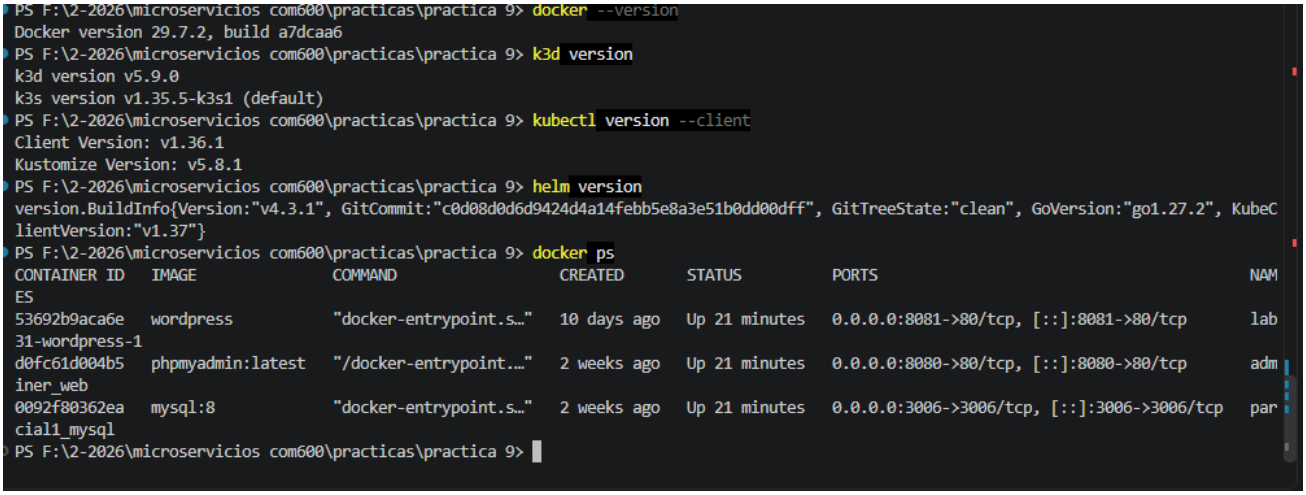
Marque con una X a medida que complete cada laboratorio.

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 0 — Verificación del entorno y creación del clúster	1, 2	1	X
Laboratorio 1 — El despliegue imperativo y por qué no sirve	3, 4	2	X
Laboratorio 2 — Manifiestos declarativos: Deployment y Service	5, 6, 7	3	X
Laboratorio 3 — Publicar hacia afuera con Ingress	8, 9	4	X
Laboratorio 4 — Configuración y secretos fuera de la imagen	10, 11, 12	5	X
Laboratorio 5 — Sondas de salud y actualización progresiva	13, 14, 15	6	X
Laboratorio 6 — Empaquetar la aplicación con Helm	16, 17	7	X
Laboratorio 7 — GitOps con Argo CD	18, 19, 20	8	

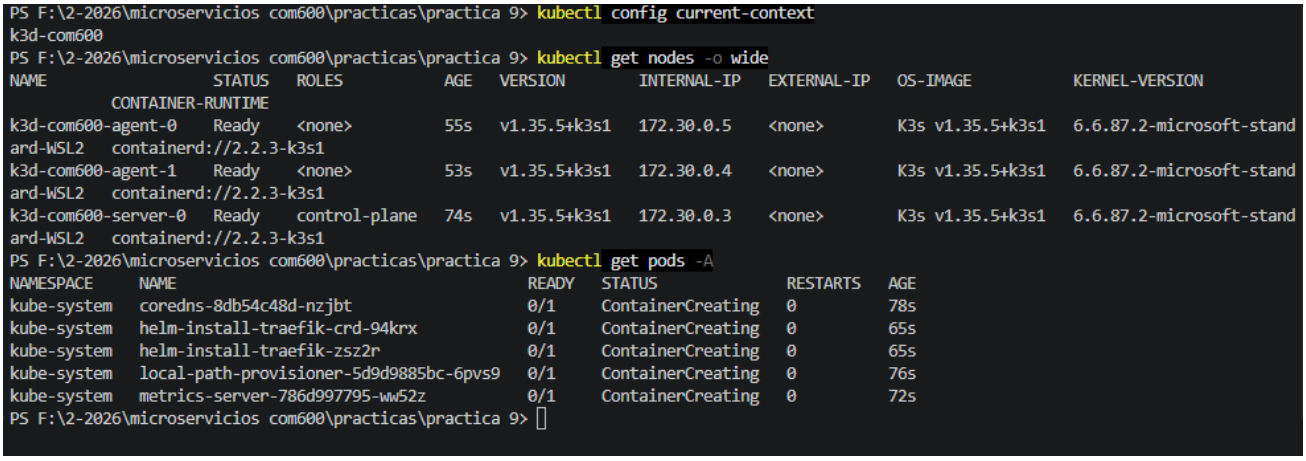
PARTE 1 — EVIDENCIAS DE LOS LABORATORIOS

Laboratorio 0 — Verificación del entorno y creación del clúster

Captura N.º 1: salida de los cinco comandos anteriores en su terminal, con las versiones visibles.



Captura N.º 2: salida de kubectl get nodes con los tres nodos en Ready y el nombre del contexto actual.

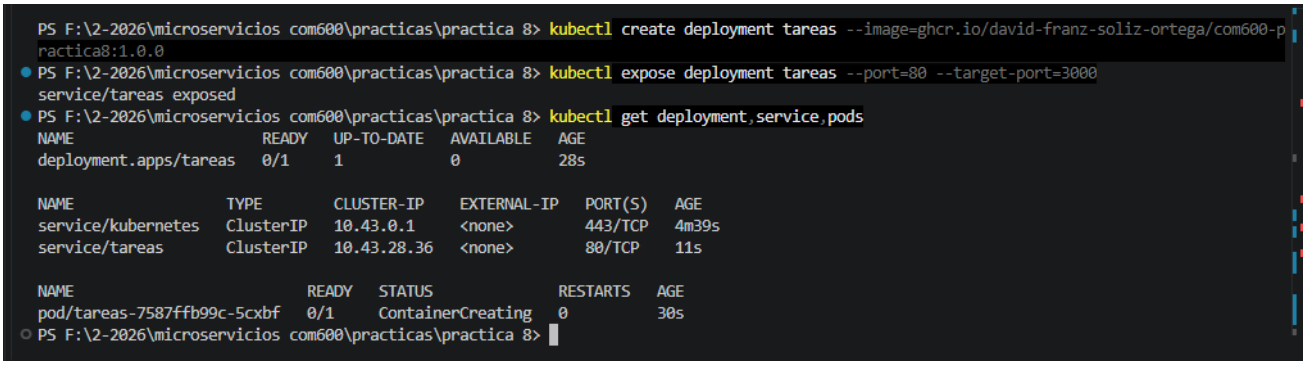


Pregunta N.º 1. Los tres nodos que acaba de ver son en realidad contenedores de Docker corriendo en su máquina. ¿Qué diferencia hay entre este clúster y uno de producción, y qué cosas de esta práctica seguirían siendo válidas allá?

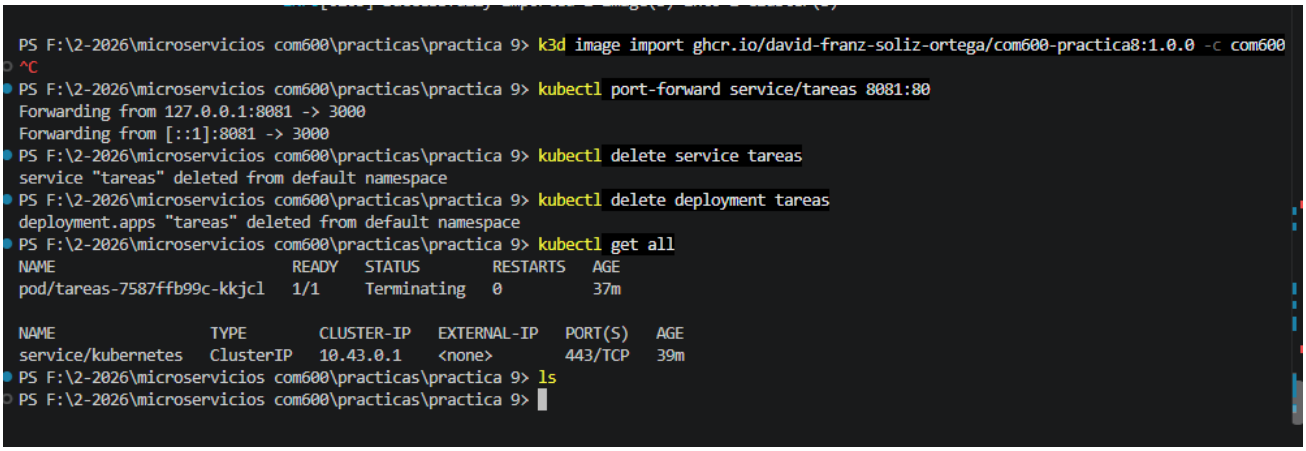
Un clúster de producción utiliza servidores reales o máquinas virtuales, con mayor disponibilidad, seguridad y capacidad. En cambio, k3d utiliza contenedores Docker en una sola máquina. Siguen siendo válidos los conceptos de Deployments, Services, Ingress, probes y Kubernetes.

Laboratorio 1 — El despliegue imperativo y por qué no sirve

Captura N.º 3: salida de kubectl get deployment,service,pods mostrando el pod en Running.



Captura N.º 4: salida de kubectl get all después del borrado y de ls mostrando que no quedó ningún archivo.

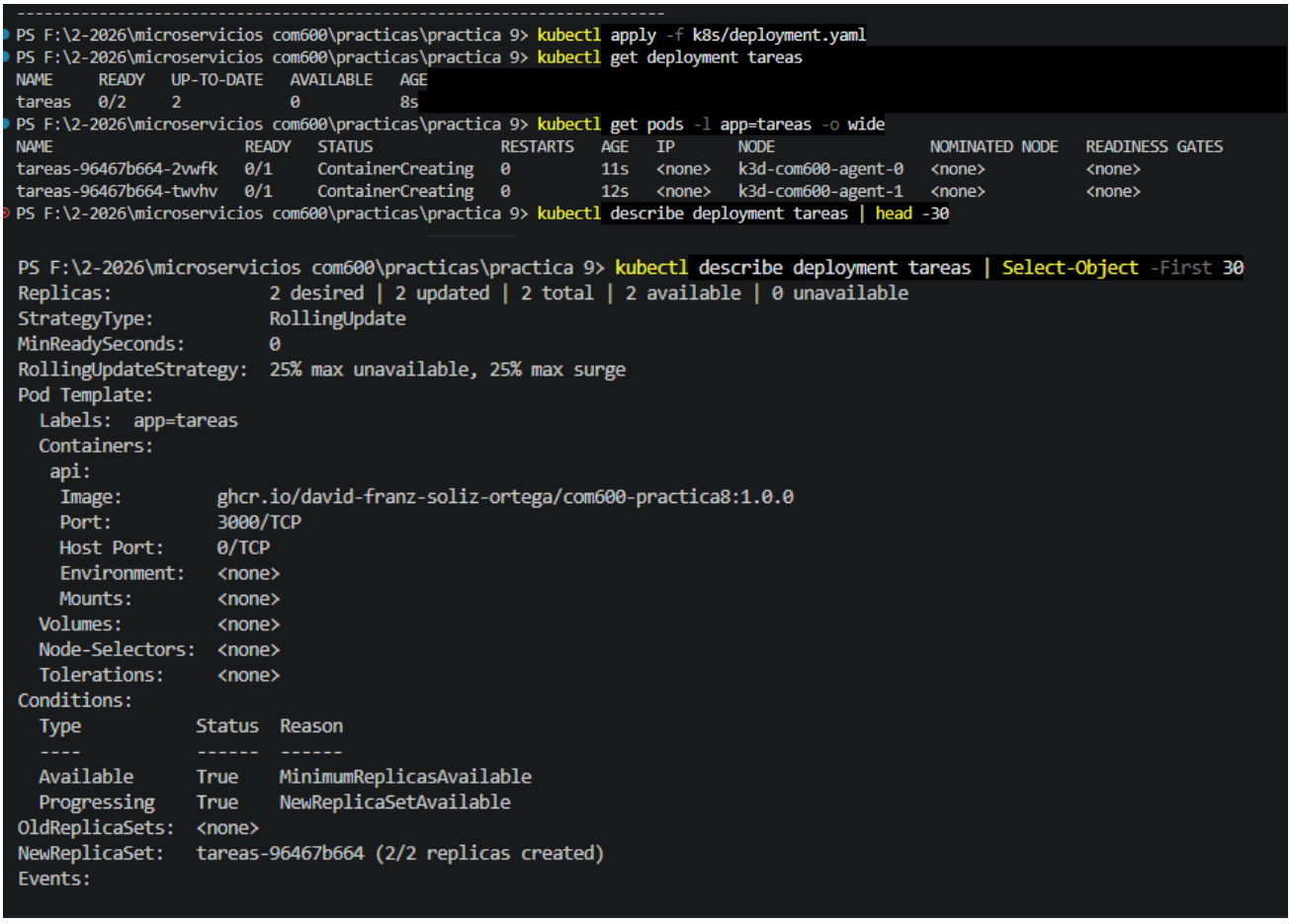


Pregunta N.º 2. Suponga que el clúster se pierde y hay que reconstruirlo tal como estaba. Con el enfoque imperativo, ¿de dónde saldría la información necesaria? Nombre dos consecuencias concretas de no tener el estado deseado escrito en un archivo versionado.

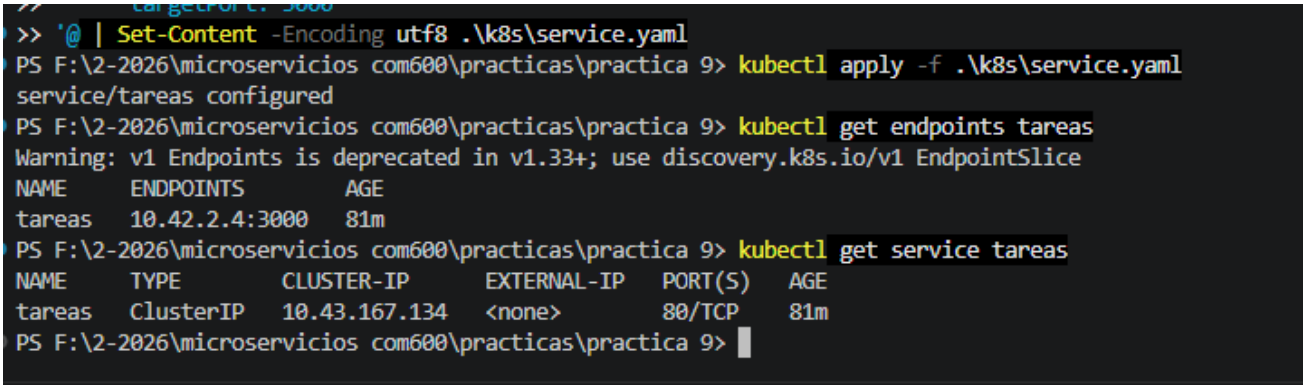
Con el enfoque imperativo, la información saldría de los comandos ejecutados y de la memoria del administrador. Sin archivos versionados, sería difícil reconstruir el entorno y no habría un historial claro de los cambios realizados.

Laboratorio 2 — Manifiestos declarativos: Deployment y Service

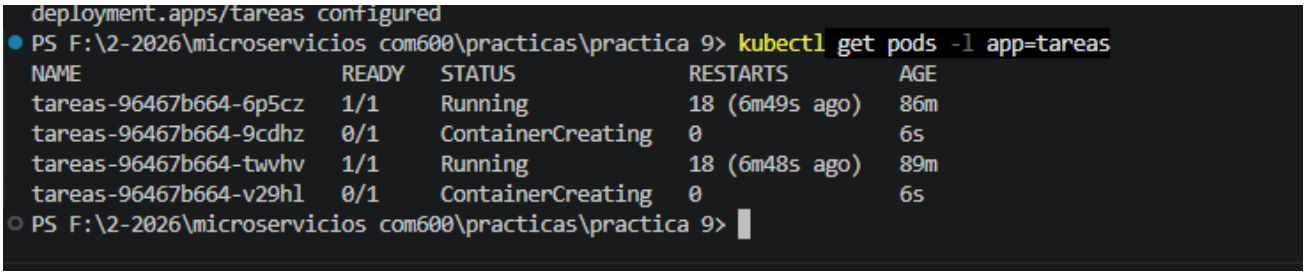
Captura N.º 5: salida de kubectl get pods -l app=tareas -o wide con las dos réplicas en Running y el nodo de cada una.



Captura N.º 6: salida de kubectl apply del Service seguida de kubectl get endpoints tareas mostrando dos direcciones.



Captura N.º 7: las cuatro réplicas en Running y la lista de endpoints del Service con cuatro direcciones.

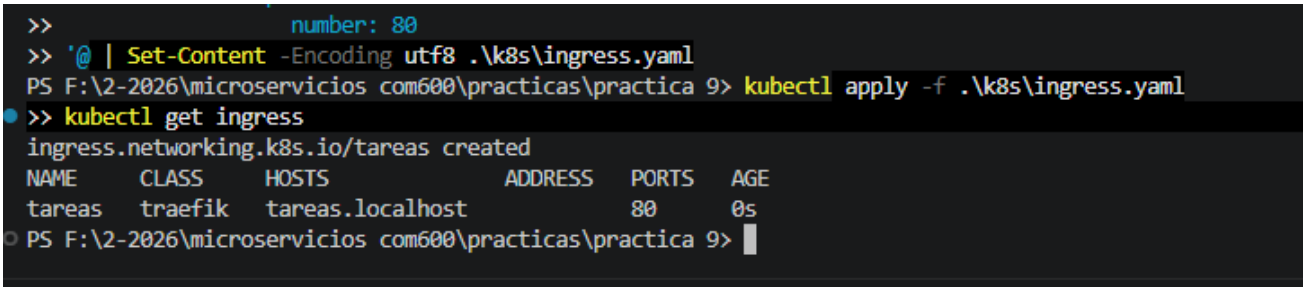


Pregunta N.º 3. Compare este laboratorio con el anterior. ¿Qué puede hacer ahora que antes era imposible, en términos de revisión por un compañero, de reconstrucción del entorno y de auditoría de quién cambió qué?

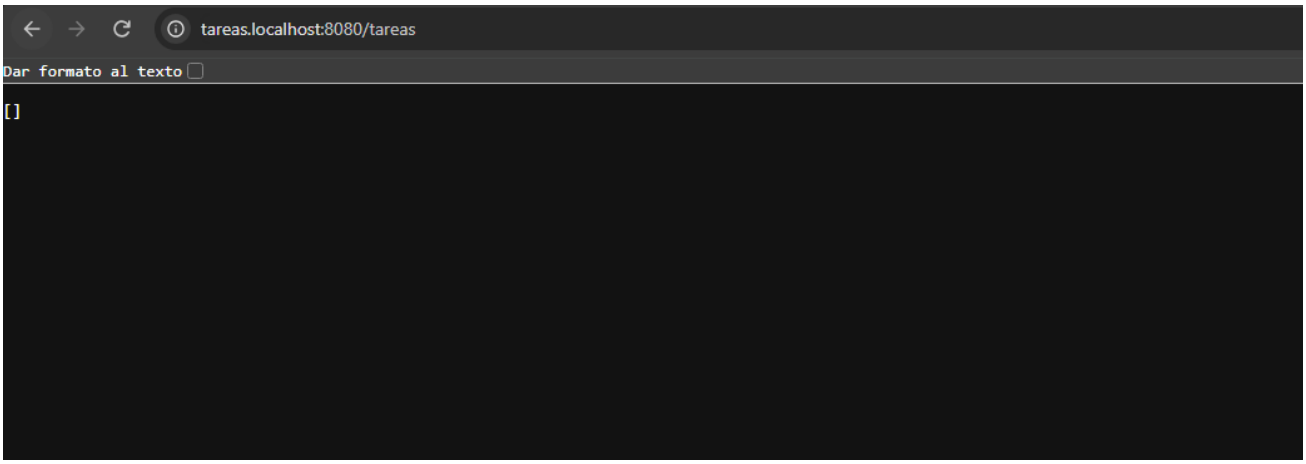
Ahora podemos revisar los cambios mediante Git, reconstruir el entorno desde los manifiestos y auditar quién cambió qué mediante el historial de commits.

Laboratorio 3 — Publicar hacia afuera con Ingress

Captura N.º 8: salida de kubectl get ingress mostrando el host tareas.localhost y la clase traefik.



Captura N.º 9: el navegador mostrando la respuesta de la API en http://tareas.localhost:8080, con la barra de direcciones visible.

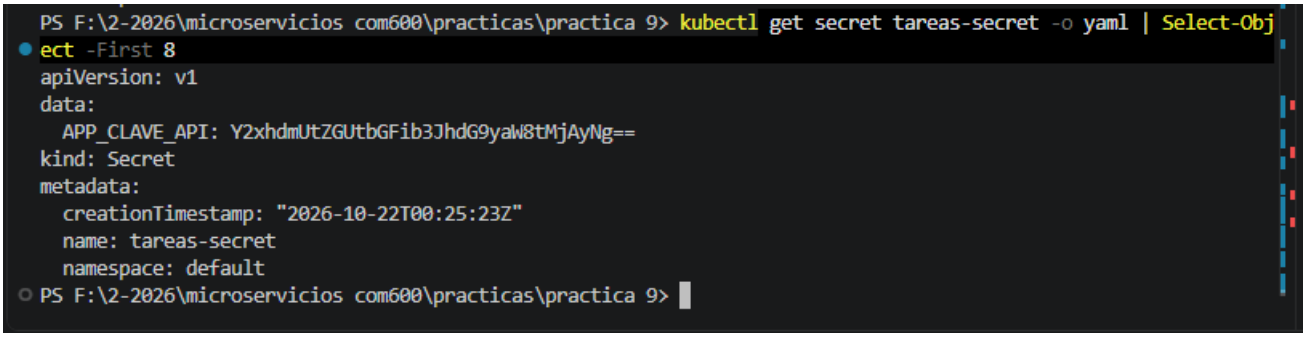


Pregunta N.º 4. Recorra el camino completo de esa petición: desde el navegador hasta el proceso que la atiende, nombrando cada pieza que interviene. ¿Qué ocurriría si el Service quedara sin endpoints mientras el Ingress sigue existiendo?

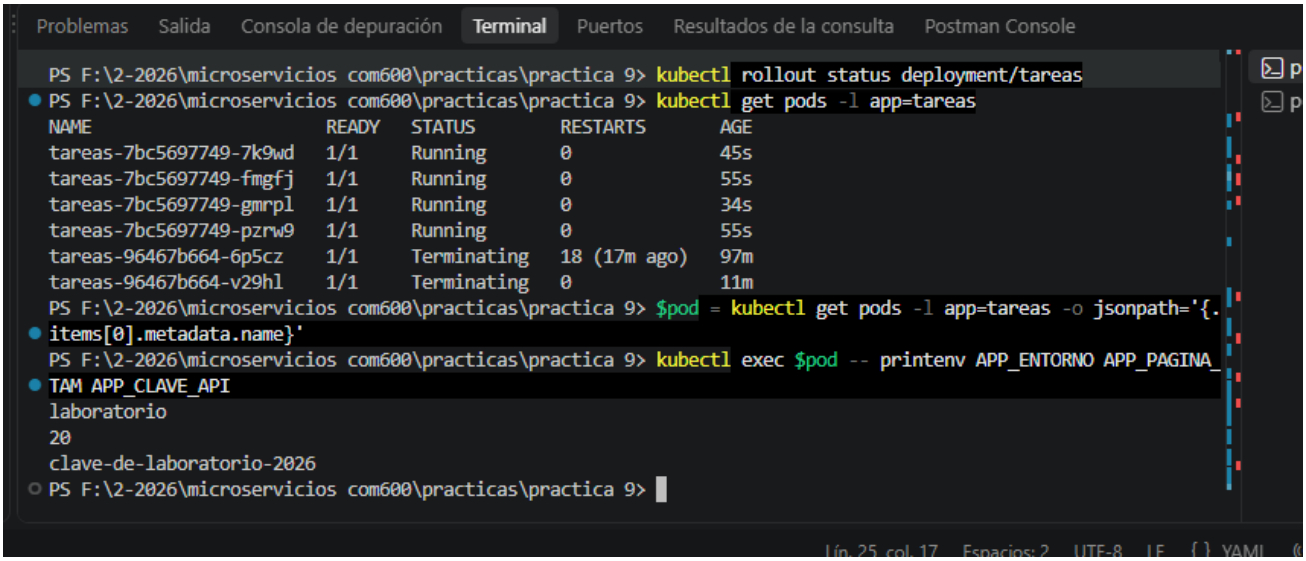
El navegador envía la petición al puerto publicado, el Ingress recibe la solicitud y la dirige al Service, que la envía a un Pod. Finalmente, el proceso de la aplicación responde. Si el Service queda sin endpoints, el Ingress sigue existiendo, pero no tiene un destino disponible y la petición falla.

Laboratorio 4 — Configuración y secretos fuera de la imagen

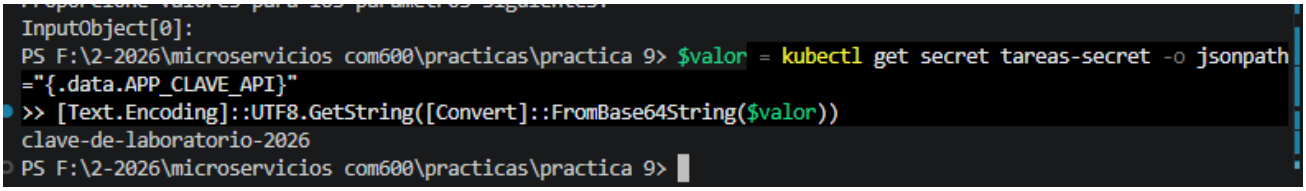
Captura N.º 10: salida de kubectl get secret -o yaml con el valor codificado visible.



Captura N.º 11: salida de kubectl exec en un pod ejecutando env y mostrando las tres variables APP_.



Captura N.º 12: la clave en texto plano recuperada desde el Secret con base64 -d.

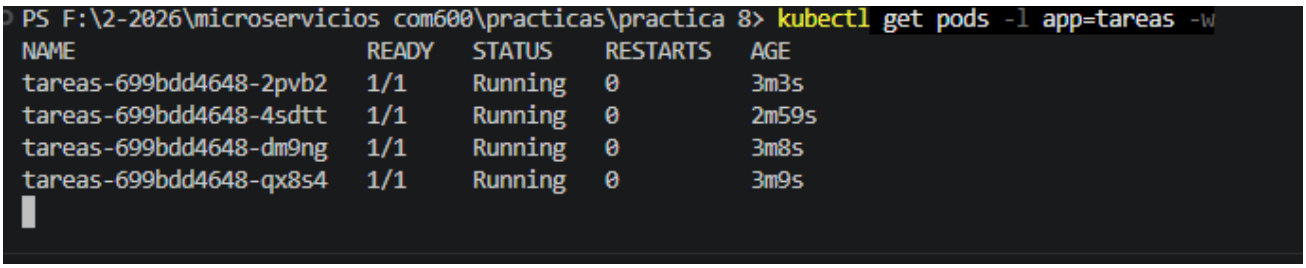


Pregunta N.º 5. Acaba de recuperar el secreto en texto plano sin ninguna credencial especial. Entonces, ¿qué problema resuelve realmente un Secret frente a escribir la clave en el Deployment, y qué haría falta agregar para que el valor esté verdaderamente protegido?

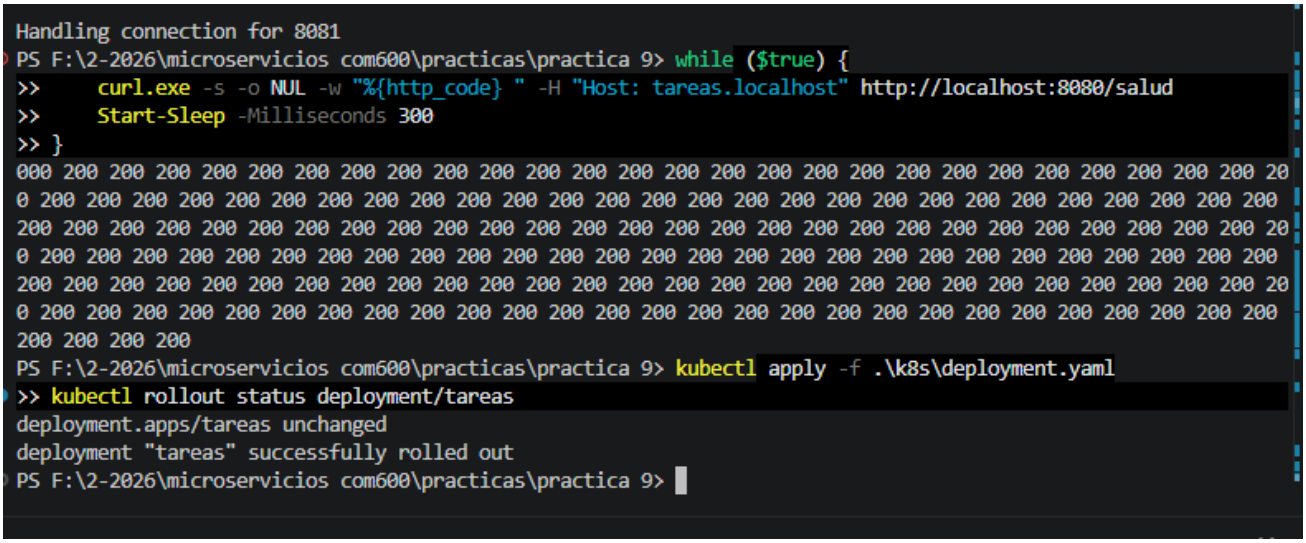
. Un Secret permite separar las claves de la configuración del Deployment, pero no las cifra automáticamente de forma segura. Para protegerlas mejor, se necesita control de acceso, cifrado de Secrets en etcd y, preferiblemente, un gestor externo de secretos.

Laboratorio 5 — Sondas de salud y actualización progresiva

Captura N.º 13: la lista de pods durante el rolling update, con réplicas de las dos versiones conviviendo.



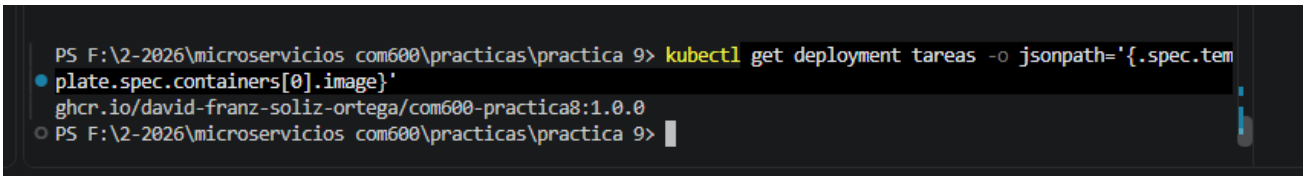
Captura N.º 14: las dos terminales: el bucle con códigos 200 continuos y el rollout ejecutándose al mismo tiempo.



Pregunta N.º 6. Explique con sus palabras qué habría pasado en ese bucle si el Deployment no tuviera readinessProbe. ¿En qué momento exacto habrían aparecido los errores y por qué?

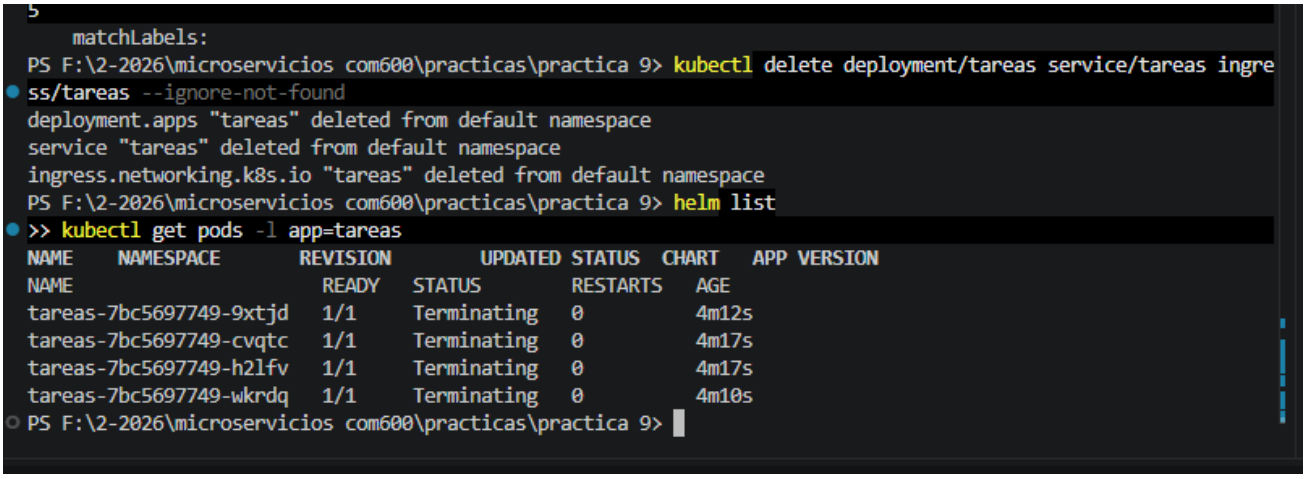
Sin readinessProbe, Kubernetes podría enviar peticiones a un Pod antes de que la aplicación esté lista. Los errores aparecerían durante el arranque porque el contenedor todavía no podría atender correctamente las solicitudes.

Captura N.º 15: el historial de revisiones y la salida de rollout undo con la imagen restaurada.

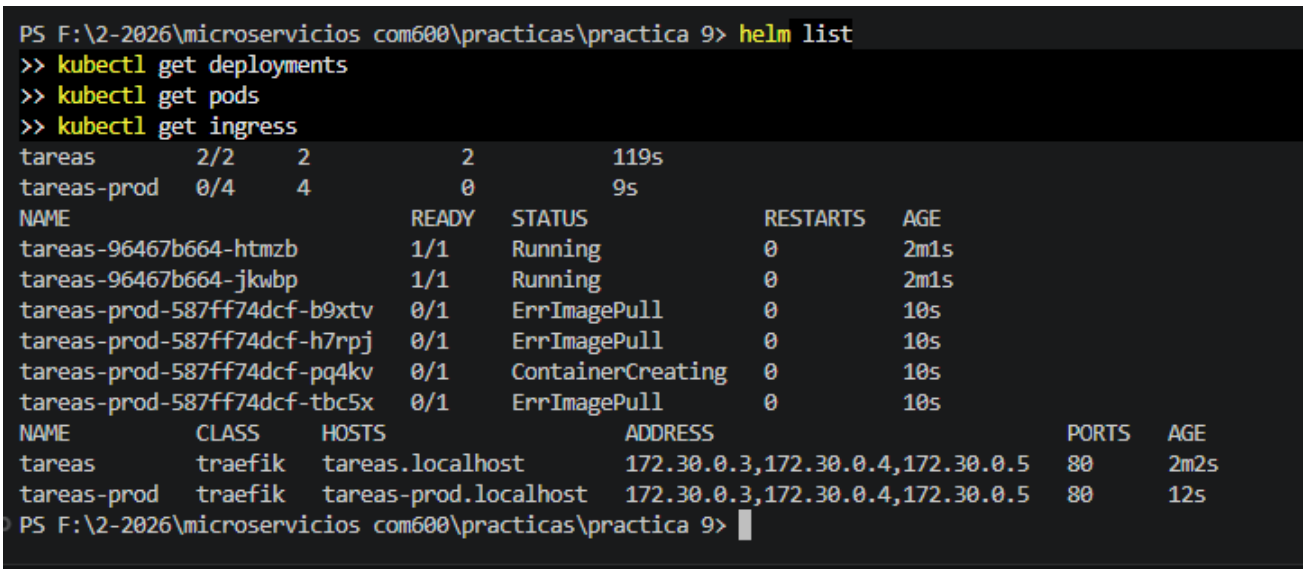


Laboratorio 6 — Empaquetar la aplicación con Helm

Captura N.º 16: salida de helm install seguida de helm list con la versión 1 desplegada y los pods en Running.



Captura N.º 17: salida de helm history con las tres revisiones y de helm list con las dos instalaciones del mismo chart.



Pregunta N.º 7. El clúster ya sabía revertir un Deployment con rollout undo. ¿Qué revierte helm rollback que kubectl rollout undo no puede revertir? Piense en un cambio que afecte al Service y al Ingress a la vez.

helm rollback puede revertir los recursos administrados por una versión anterior del chart, como Deployment, Service e Ingress. kubectl rollout undo solo revierte la revisión del Deployment, no los cambios de los demás recursos.

Laboratorio 7 — GitOps con Argo CD

Captura N.º 18: la interfaz de Argo CD con la aplicación tareas en estado Synced y Healthy, mostrando el árbol de recursos.

[Pegue aquí la captura de pantalla]

Captura N.º 19: el commit en GitHub y, al lado, los pods del clúster reflejando el nuevo número de réplicas.

[Pegue aquí la captura de pantalla]

Captura N.º 20: la interfaz de Argo CD mostrando el estado OutOfSync y, a continuación, el retorno a Synced con las réplicas del repositorio.

[Pegue aquí la captura de pantalla]

Pregunta N.º 8. Un compañero afirma que GitOps sirve para desplegar más rápido. Argumente por qué esa no es su ventaja principal y nombre dos problemas concretos de operación que sí resuelve. ¿Qué le habría pasado al rollout undo del Laboratorio 5 en este esquema?

La principal ventaja de GitOps no es desplegar más rápido, sino mantener el clúster sincronizado con el estado declarado en Git. Resuelve problemas como la configuración manual inconsistente y los cambios no registrados. Si se ejecutara rollout undo directamente, Argo CD detectaría la diferencia y, con selfHeal activo, intentaría restaurar el estado definido en Git.

Cierre de la Parte 1

Responda brevemente antes de entregar:

¿Qué parte de la práctica le resultó más difícil y por qué?

La parte de argocd

¿Qué error tuvo que resolver por su cuenta y cómo lo resolvió?

La parte de chart tuve que ver como corregir errores con algunos comandos

¿Qué haría distinto si tuviera que repetir esta práctica?

Tener instalado winget y funcionando en la terminal

Declaración de autoría: Declaro que las capturas presentadas fueron tomadas en mi propio equipo y que las respuestas son de mi autoría. Entiendo que presentar evidencias de otra persona anula la Parte 1 de esta práctica.

_____ _____

Firma del estudiante Fecha